

BanGaNNet: A Lightweight Deep Learning Framework for Bangla Handwritten Text Recognition

Argho Deb Das

Computer Science and Engineering
Northern University Bangladesh
Dhaka-1230, Bangladesh
atdeb727@gmail.com

Dabbrata Das

Computer Science and Engineering
Uttara University
Dhaka-1230, Bangladesh
dasdabbrata@gmail.com

Farhan Sadaf

Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna-9203, Bangladesh
imfarhansadaf@gmail.com

Abstract—Bangla Handwritten Text Recognition (BHTR) presents a significant challenge in Optical Character Recognition (OCR) owing to the script's complex structure, extensive set of compound characters, and high variability in handwriting styles. Addressing this issue is critical for applications including document digitalization, postal automation, and mobile text input in Bangla, the world's seventh most spoken language. Existing deep learning algorithms, such as CNN, CRNN, and Gated-CNN based models, achieve high accuracy but are computationally expensive and inappropriate for resource-constrained devices. To address these limitations, we present BanGaNNet, a lightweight end-to-end architecture that integrates gated CNN blocks, batch normalization, and stacked bidirectional LSTMs trained with Connectionist Temporal Classification (CTC) loss for alignment-free sequence modeling. When tested on the BanglaWriting dataset, BanGaNNet achieves state-of-the-art performance with 12.37% Character Error Rate and 34.16% Word Error Rate while requiring only 1.32M parameters. These contributions demonstrate BanGaNNet's effectiveness and resource efficiency, with prospective applications in real-time mobile OCR, low-power embedded devices, and large-scale Bangla document digitization.

Keywords— Bangla Handwritten Text Recognition (BHTR), HTR, Deep learning, Gated CNN, CTC Loss

I. Introduction

Bangla Handwritten Text Recognition (BHTR) remains a difficult and unsolved problem in Optical Character Recognition (OCR) due to the script's structural complexity and wide variations in handwriting styles. The Bangla script consists of a large number of basic and compound characters, ligatures, and visually subtle diacritical marks that frequently change their shape depending on the writer's stroke pattern, curvature, or character joining behavior. These aspects make segmentation and recognition extremely challenging, especially in unconstrained settings. Although printed Bangla OCR has advanced considerably, handwritten recognition still struggles to achieve reliable performance in practical scenarios such as document digitization, postal automation, and large-scale archival transcription, as highlighted in prior studies [1].

Earlier handwriting recognition methods depended on handcrafted features and statistical classifiers, but these approaches failed to capture the rich visual structure and contextual dependencies inherent in Bangla handwriting. With the emergence of deep learning, CNN- and LSTM-based architectures have significantly improved recognition accuracy by learning hierarchical representations directly from data. However, existing BHTR solutions continue to suffer

from several limitations. Many state-of-the-art deep models are computationally heavy, containing millions of parameters that restrict their deployment in low-resource or embedded systems. Furthermore, because annotated Bangla handwriting datasets are relatively limited in scale, these models often exhibit overfitting and weak generalization, especially when trained without strong normalization or regularization mechanisms. Another persistent limitation is that several architectures focus primarily on spatial features while inadequately capturing sequential dependencies across the word image, leading to errors in complex or visually ambiguous handwriting. Character-level recognition pipelines further amplify these issues, as segmentation errors caused by touching characters or irregular spacing degrade end-to-end accuracy.

This work addresses these gaps by focusing on word-level BHTR, a more practical and robust formulation for real-world use cases. We introduce BanGaNNet, a lightweight and efficient end-to-end architecture designed to improve recognition accuracy while drastically reducing computational overhead. BanGaNNet integrates gated convolutional layers that selectively enhance informative features and suppress noise, along with Batch Normalization to stabilize training and improve generalization. A Bidirectional LSTM-based Hybrid Text Recognition Decoder (HTRd) captures long-range contextual dependencies across the word image, and the use of Connectionist Temporal Classification (CTC) loss [2] enables alignment-free sequence prediction without requiring character segmentation. The full pipeline incorporates grayscale preprocessing and probabilistic augmentation to handle real-world handwriting variations effectively.

Experiments conducted on the BanglaWriting dataset [3] demonstrate that BanGaNNet achieves state-of-the-art performance in both Character Error Rate (CER) and Word Error Rate (WER) while using only 1.32M parameters, making it significantly lighter than existing architectures in this domain. The combination of compact design, normalized gated feature extraction, and efficient sequential modeling allows BanGaNNet to operate effectively in low-resource or embedded environments, offering a practical and deployable solution for real-world Bangla OCR applications.

II. Related Works

Handwritten Bangla text recognition has seen a wide range of deep learning approaches in recent years. These methods can be broadly categorized based on the granularity of recognition: character-level and word-level recognition.

A. Character-Level Recognition

Early works primarily focused on recognizing isolated Bangla characters using deep convolutional neural networks [4] trained

on segmented datasets. Alom et al. [5] evaluated various CNN architectures, including VGG, All Convolutional Network, NiN, ResNet, FractalNet, and DenseNet on the CMATERdb [6] dataset, achieving high accuracy (up to 98.18%). Similarly, Saha et al. [7] proposed an ensemble framework combining Inception, ResNet, and DenseNet, resulting in strong generalization performance across multiple datasets.

Chakraborty et al. [8] developed a deep CNN (DCNN) based model for Bangla handwritten character recognition, achieving up to 93.07% accuracy on their created dataset. They compared simple CNN, DCNN, and VGG-16 architectures across multiple datasets, demonstrating the effectiveness of deep, optimized models for feature extraction and classification. Ghosh et al. [9] evaluated state-of-the-art pre-trained CNN architectures for recognizing 231 Bangla handwritten characters using the CMATERdb dataset. Inception-ResNetV2 [10] achieved the highest accuracy of 96.99%, followed by DenseNet121 and InceptionNetV3. An ensemble of these models reached 97.69% accuracy but was computationally expensive. Considering performance and efficiency, InceptionResNetV2 was identified as the best balance for Bangla character recognition.

While these methods are effective for recognizing segmented or isolated characters, they often struggle with unconstrained handwritten text due to the lack of contextual understanding and frequent segmentation errors.

B. Word-Level Recognition

To overcome limitations in segmentation and contextual modeling, several approaches have focused on word-level handwritten recognition. Sadaf et al. [11] conducted a benchmarking study focusing on Bangla word recognition, evaluating CRNN and Gated-CNN models with pre-trained CNN backbones (MobileNet, Xception, DenseNet) and recurrent layers (LSTM, GRU).

Hossain et al. [12] proposed a Bangla handwritten word recognition system based on Convolutional Neural Networks (CNN), addressing the challenges of segmenting multi-zoned characters in Bangla script. Their method introduced a robust segmentation and character merging approach, achieving 84% accuracy at the character level and 82% accuracy at the word level, demonstrating its effectiveness in structured recognition tasks. Akter et al. [13] addressed limited Bangla handwritten word data by generating synthetic word images from characters and training a BiLSTM-CTC model. Their system achieved 39% WER and 92% accuracy on non-overlapped data, and 63% WER with 83% accuracy on overlapped data. Roy et al. [14] proposed a Hidden Markov Model (HMM)-based technique utilizing zone-wise segmentation. Although effective, this method relies heavily on handcrafted features, reducing its adaptability to complex handwriting variations.

Despite these developments, challenges such as computational complexity, generalization to varied handwriting styles, and dependency on segmentation persist. End-to-end word-level models provide robustness but often demand further optimization for real-world applications.

III. Proposed Methodology

A. Dataset

For this work, we used the BanglaWriting dataset [3], which is a rich resource for recognizing Bangla handwritten writing. The collection contains 21,234 word instances and 32,787 character samples taken from 260 single-page handwritten texts. These papers contain 5,470 distinct Bangla words, each annotated with its appropriate Unicode representation and accurate word-level bounding boxes.

To assure data quality, we removed defective samples and word occurrences with more than ten characters, yielding a revised dataset of 20,736 word pictures. These graphics represent a character set of 90 different Bangla symbols, including consonants, vowels, numerals, matras, and popular punctuation signs. Fig. 1 shows the whole

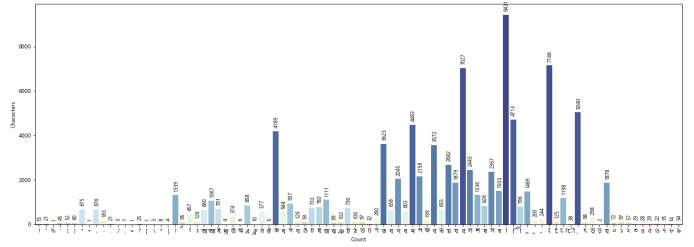


Figure 1. Character-set used for classification from BanglaWriting dataset.

set of characters used for classification. The diversity and structure of this dataset make it ideal for developing and testing word-level handwritten text recognition models for the Bangla script.

B. Preprocessing

Image and text preprocessing techniques were used to prepare the BanglaWriting dataset [3] for training. At the picture level, each handwritten phrase was scaled to a constant 256×64 pixels, maintaining aspect ratio using zero-padding (Fig. 2). This ensured consistent input proportions while preserving handwritten shapes.

Word sequences were created at the text level using a 90-symbol vocabulary that included letters, digits, and punctuation. To compensate for the CTC loss, a blank token was added whenever consecutive duplicate symbols appeared, guaranteeing proper alignment between input frames and label sequences. Table I provides representative examples of this approach.

These steps normalized visual inputs and textual labels, resulting in a balanced dataset suited for end-to-end training.

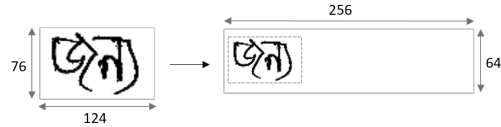


Figure 2. Example of dataset image preprocessing with padding.

Table I
Preprocessing examples of ground truth texts.

Word from Dataset	Word after preprocessing
উপর	উ, প, র
ন্যায্য	ন, ্, য, া, য, ্, য
কানন	ক, া, ন, , ন
গ্রীষ্ম	গ, ্, র, ী, ষ, ্, ম
প্রশান্তি	প, ্, র, শ, া, ন, ্, ত, ি
স্কুল	স, ্, ক, ু, ল

C. Augmentation

For better generalization, various transformations were used during training, as seen in Fig. 3. Geometric alterations (rotation, scaling, translation, and distortion) were combined with photometric variations (brightness adjustment, Gaussian blur, noise injection, sharpening, and pixel dropout). Each modification was carried out stochastically with a global probability of 0.6, ensuring that augmented samples retained readability while introducing adequate unpredictability. This combination improved model robustness to handwriting style, stroke abnormalities, and imaging artifacts.



Figure 3. Example of an augmented text image demonstrating various transformations applied during training.

D. Optical Models

The optical model is at the heart of the handwriting recognition system, as it learns the mapping between the input image and the matching character sequence. In this paper, we look at pre-trained convolutional encoders, proven CRNN-based architectures, and a novel lightweight model to see how they perform on Bangla handwritten word recognition.

1) *Pre-trained CNN Backbones*: We used multiple pre-trained convolutional neural network (CNN) models as feature extractors to make use of existing visual representations including Xception [15], MobileNet [16], DenseNet121 [17]. Each of these CNN backbones extracts high-level feature maps from an input image. The generated features are then passed into a recurrent sequence modeling module, which predicts character sequences.

2) *CRNN-Based Architecture (Puigcerver)*: For benchmarking purposes, we incorporate Puigcerver et al.'s CRNN design. [18] uses convolutional layers for feature extraction and bidirectional LSTMs for sequence modeling.

3) *Gated-CRNN Architecture (Flor)*: We also test the Gated-CRNN model proposed by Flor et al. [19] uses Gated CNN layers, followed by BiLSTM and dense layers, to efficiently recognize handwritten text with less parameters.

4) *Proposed Architecture*: We provide a lightweight and efficient deep learning model designed specifically for recognizing Bangla handwritten texts. The design employs a standard encoder-decoder pattern, integrating convolutional feature extraction and recurrent sequence modeling.

Fig. 4 illustrates the total architecture. It starts with four convolutional blocks. Each block starts with a 3x3 convolutional layer, followed by PReLU activation, Batch Normalization (with re-normalization), and a Full Gated Convolutional layer. After the first three blocks, MaxPooling is used to lower spatial dimensions incrementally. Two dropout layers are included after the third and fourth convolutional stages to prevent overfitting.

Following the convolutional encoder, the feature maps are reshaped into a two-dimensional sequence and sent through two stacked Bidirectional LSTM layers, each with 128 hidden units. To represent intermediate sequences, a fully connected dense layer of 256 units is added between the LSTM layers. The final output layer is a softmax classifier that generates probability distributions for 78 target character classes.

The model is trained using Connectionist Temporal Classification (CTC) loss, which allows for alignment-free training by learning to correlate a series of feature vectors with the target transcription. With only 1.32 million trainable parameters, the suggested model is computationally efficient while maintaining accuracy.

E. Loss Calculation and Decoding

In handwriting recognition, the alignment of input image features and output text sequences is frequently uncertain. To solve this, we use the Connectionist Temporal Classification (CTC) loss function, which enables training without explicit alignment by learning to map variable-length sequences.

1) *CTC Loss*: The Connectionist Temporal Classification (CTC) loss [20] allows the model to predict character sequences without explicitly delineated character boundaries. It adds a blank token to handle flexible alignments, repeated characters, and variable-length

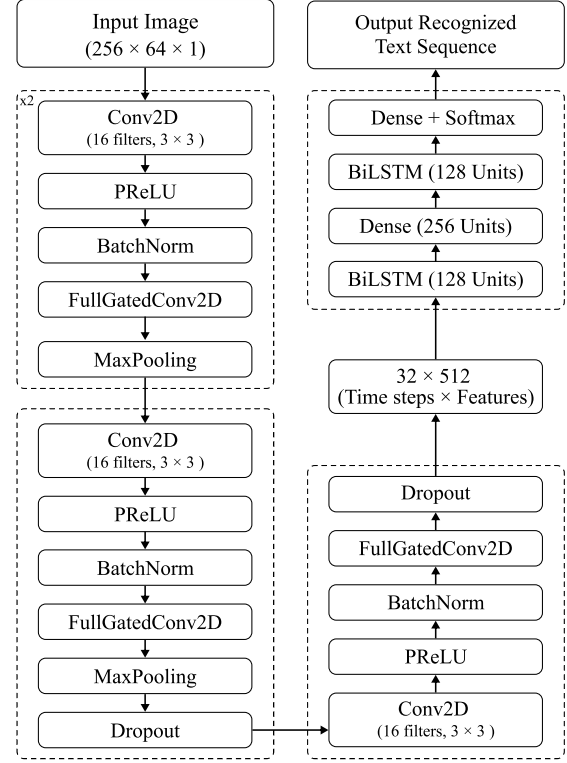


Figure 4. Block diagram of the proposed architecture.

outputs. In Equation 1, y represents the anticipated probability distribution for output characters, whereas t represents the target sequence. The term $P(t | y)$ sums the probabilities for all valid alignments between forecasts and targets. This design enables the model to learn the most likely transcription path without relying on frame-level labeling, resulting in robust sequence learning even in the absence of explicit alignment information.

$$\mathcal{L}_{CTC} = -\log P(t | y) \quad (1)$$

Consider the following example of CTC decoding for Bangla handwriting recognition.

CTC decoding follows two main rules:

- Repeated characters are combined into a single instance unless separated by a blank token.
- All blank tokens (e.g., $\langle /B \rangle$) are eliminated from the final output.

Suppose the model predicts the following sequence over time:

Prediction: ক, ক, $\langle /B \rangle$, া, $\langle /B \rangle$, ন, $\langle /B \rangle$, ন, $\langle /B \rangle$

Output after CTC decoding: 'কানন'

Ground truth: 'কানন'

In this example, the model accurately predicts the letter sequence, adding blank tokens between repeated instances of 'ন'. This permits both characters to be included in the final transcript. The use of

blanks ensures appropriate alignment without the need for explicit segmentation, which is especially important in complex scripts such as Bangla. Although this approach allows for greater flexibility in learning alignments, it can also produce problems if blank tokens or repeated characters are placed incorrectly.

2) *Decoding Strategies*: Following CTC training (Equation 1), the model outputs a probability distribution over characters at each time step. To obtain the final sequence, two decoding methods are applied:

- **Greedy Search** selects the most probable character at each step (Equation 2). It is fast but may overlook global sequence likelihood. Duplicates and blank tokens are removed to form the output.
- **Beam Search** [21] explores multiple candidate sequences and chooses the one with the highest cumulative log probability (Equation 3). Though slower, it usually yields better accuracy, especially for long or ambiguous inputs.

$$\hat{c}_t = \arg \max_c y_{t,c} \quad (2)$$

$$S(c) = \sum_{t=1}^T \log y_{t,c_t} \quad (3)$$

F. System Overview

The proposed BHTR pipeline integrates preprocessing, augmentation, feature extraction, sequence modeling, and decoding into a unified framework (Fig. 5). During training, preprocessed word images are passed through the optical model to extract spatial features, followed by a recurrent sequence model that produces time-distributed character probabilities. The system is optimized using CTC loss (Section 3.4.1), enabling alignment-free sequence learning. At inference, probability matrices are decoded via Greedy or Beam Search to generate final predictions, with performance evaluated using CER and WER. This modular design ensures end-to-end trainability while allowing flexibility in model, decoding, and augmentation choices.

IV. Experimental Analysis

A. Experimental Setup

All models were implemented using TensorFlow [22] and NumPy [23] libraries. The Albumentations library [24] was used for data augmentation with various geometric and photometric changes (details in Section 3.2). Training was led by the Connectionist Temporal Classification (CTC) loss function [20], which is suitable for unsegmented sequence labeling. The dataset was partitioned randomly into three subsets: 18,663 for training, 829 for validation, and 1,244 for testing. To avoid overfitting, early stopping was used, which terminated training if the validation loss did not improve after five consecutive epochs. Furthermore, the learning rate was automatically lowered when the model's performance plateaued during training. Models were trained with the Adam optimizer with a learning rate of 0.001 and a batch size of 32. We employed Vanilla Beam Search [21] with a beam width of 10, which allowed the model to explore several candidate sequences and enhance identification accuracy compared to greedy decoding. All tests were carried out on Google Colab [25] with an NVIDIA Tesla T4 GPU, which offered adequate computational resources for training and evaluation.

B. Evaluation Metrics

The performance of recognition models is measured using Character Error Rate (CER) and Word Error Rate (WER), both derived from the Levenshtein distance [26], which counts the minimum insertions, deletions, and substitutions needed to align predictions with ground truth.

Table II
Performance comparison and number of parameters of different models.

Model	Validation		Test		Parameters
	CER%	WER%	CER%	WER%	
Xception	27.46	60.67	28.46	62.86	20,872,045
MobileNet	39.90	75.39	42.08	76.69	1,945,501
DenseNet121	37.37	74.43	38.99	78.79	4,695,069
Puigcerver	15.89	38.96	16.22	37.18	8,277,917
HTR-Flor	14.27	36.79	13.99	35.65	774,158
Proposed BanGaNet	12.78	36.67	12.37	34.16	1,322,686

- **CER** is the ratio of character-level errors (substitutions S , deletions D , insertions I) to the total number of characters N in the reference text:

$$\text{CER} = \frac{S + D + I}{N} \quad (4)$$

- **WER** is defined similarly at the word level, where N is the total number of words in the reference text:

$$\text{WER} = \frac{S + D + I}{N} \quad (5)$$

V. Results and Analysis

To assess the performance of the proposed architecture and compare it to existing models, we publish the test set's Character Error Rate (CER) and Word Error Rate (WER). We also examine the number of trainable parameters in each model to determine computational efficiency.

A. Performance Comparison

Table II summarizes the CER and WER findings for all models in the validation and test sets of the BanglaWriting dataset [3]. The model has the lowest error rates in both the validation and test stages, demonstrating its generalizability and efficiency in identifying Bangla handwritten text. In addition to quantitative indicators, we examined the selected models' training and validation loss curves to better understand their convergence behavior. Fig. 6 demonstrates that the HTR-Flor, Puigcerver, MobileNet, and Proposed BanGaNet models have different levels of training stability and generalization.

B. Parameter Analysis

Table II also displays the number of trainable parameters per model. While deeper designs like as Xception provide reasonable performance, they have a much greater parameter footprint. In comparison, the suggested model achieves competitive accuracy while using a substantially smaller size.

The suggested BanGaNet model achieves the optimal mix of accuracy and computational cost, making it ideal for use in real-world Bangla handwritten recognition applications.

VI. Limitations

While the suggested system performs well, there are a few drawbacks that indicate areas for future enhancement. One significant constraint is the distribution of the training dataset. Fig. 1 shows that the BanglaWriting dataset's [3] character set is not equally distributed. Certain characters occur more frequently than others, potentially biasing the model toward overrepresented classes. This imbalance may impair performance with rare or complicated characters. Furthermore, the current approach mainly focuses on word-level recognition, assuming accurate word segmentation is available. In actual applications, faults in segmentation or line-level input might impair overall system accuracy.

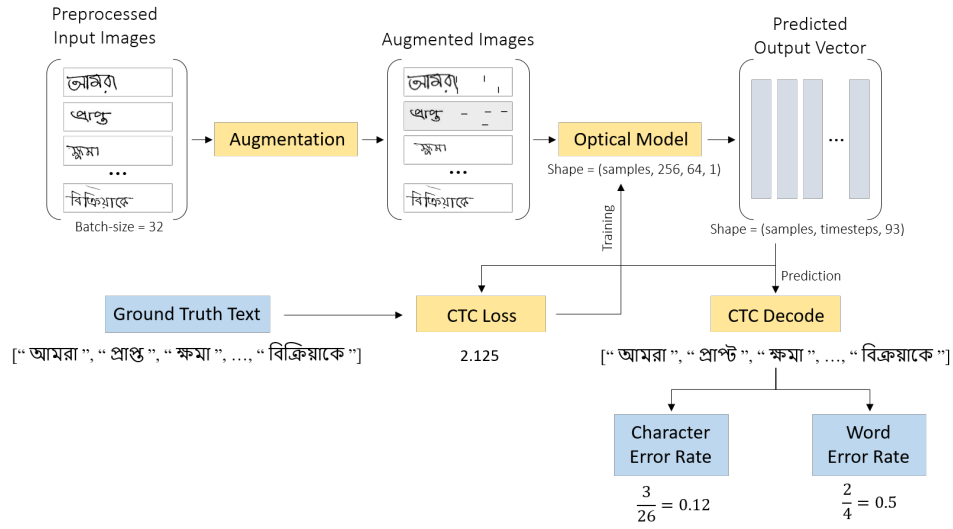


Figure 5. Stages of training from start to finish for the planned architecture.

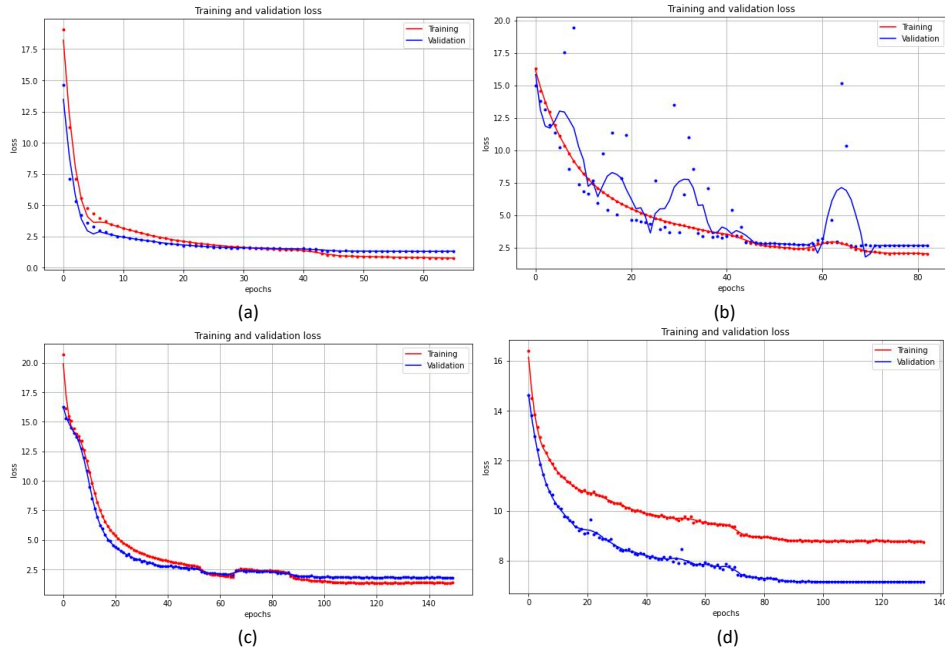


Figure 6. Training and validation loss curves for selected models: (a) BanGaNet (b) Puigcerver (c) HTR-Flor (d) Mobilenet.

VII. Conclusion

This study addressed Bangla Handwritten Text Recognition (BHTR) using deep learning-based models, including both benchmarks and the proposed lightweight BanGaNet. Evaluated on the challenging BanglaWriting dataset [3], BanGaNet achieved state-of-the-art performance with substantially fewer parameters, obtaining the lowest CER (12.37%) and WER (34.16%) while demonstrating strong convergence and generalization. Batch Normalization and Transfer Learning further enhanced stability and accuracy across models. These results confirm the suitability of BanGaNet for real-world, resource-constrained applications such as mobile OCR and embedded Bangla reading aids. Future work will extend recognition to lines and paragraphs, address compound character reconstruction, and integrate language models for context-aware decoding.

References

- [1] R. Avyodri, S. Lukas, and H. Tjahyadi, "Optical character recognition (ocr) for text recognition and its post-processing method: A literature review," in *2022 1st International Conference on Technology Innovation and Its Applications (ICTIIA)*, pp. 1–6, 2022.
- [2] R. Takashima, S. Li, and H. Kawai, "Ctc loss function with a unit-level ambiguity penalty," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5909–5913, 2018.
- [3] M. F. Mridha, A. Q. Ohi, M. A. Ali, M. I. Emon, and M. M. Kabir, "Banglawriting: A multi-purpose offline bangla handwriting dataset," *Data in Brief*, vol. 34, p. 106633, 2021.
- [4] S. Bhattacharjee and A. R. Nath, "Sports video classification using vision transformer: A deep learning based approach," in *2025 Inter-*

- national Conference on Quantum Photonics, Artificial Intelligence, and Networking (QPAIN)*, pp. 1–6, 2025.
- [5] M. Z. Alom, P. Sidike, M. Hasan, T. M. Taha, and V. K. Asari, “Handwritten bangla character recognition using the state-of-art deep convolutional neural networks,” *CoRR*, vol. abs/1712.09872, 2017.
 - [6] M. M. Rahman, “Bangla handwritten character db cmaterdb 3.1.2,” 2023.
 - [7] C. Saha and M. M. Rahman, “Banglanet: Bangla handwritten character recognition using ensembling of convolutional neural network,” 2024.
 - [8] P. Chakraborty, A. Islam, M. Abu Yousuf, R. Agarwal, and T. Choudhury, “Bangla handwritten character recognition using convolutional neural network,” in *Machine Intelligence and Data Science Applications* (V. Skala, T. P. Singh, T. Choudhury, R. Tomar, and M. Abul Bashar, eds.), (Singapore), pp. 721–731, Springer Nature Singapore, 2022.
 - [9] T. Ghosh, M. Abedin, M. Al Banna, N. Mumenin, and M. Yousuf, “Performance analysis of state of the art convolutional neural network architectures in bangla handwritten character recognition,” *Pattern Recognition and Image Analysis*, vol. 31, pp. 60–71, 01 2021.
 - [10] D. Das, A. D. Das, and F. Sadaf, “Enhanced encoder-decoder architecture for accurate monocular depth estimation,” *The Visual Computer*, vol. 41, no. 12, pp. 9487–9508, 2025.
 - [11] F. Sadaf, S. M. Taslim Uddin Raju, and A. Muntakim, “Offline bangla handwritten text recognition: A comprehensive study of various deep learning approaches,” in *2021 3rd International Conference on Electrical Electronic Engineering (ICEEE)*, pp. 153–156, 2021.
 - [12] M. T. Hossain, M. W. Hasan, and A. K. Das, “Bangla handwritten word recognition system using convolutional neural network,” in *2021 15th International Conference on Ubiquitous Information Management and Communication (IMCOM)*, pp. 1–8, 2021.
 - [13] M. S. Akter, H. Shahriar, A. Cuzzocrea, N. Ahmed, and C. Leung, “Handwritten word recognition using deep learning approach: A novel way of generating handwritten words,” in *2022 IEEE International Conference on Big Data (Big Data)*, p. 5414–5423, IEEE, Dec. 2022.
 - [14] P. P. Roy, P. Dey, S. Roy, U. Pal, and F. Kimura, “A novel approach of bangla handwritten text recognition using hmm,” in *2014 14th International Conference on Frontiers in Handwriting Recognition*, pp. 661–666, 2014.
 - [15] F. Chollet, “Xception: Deep learning with depthwise separable convolutions,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1251–1258, 2017.
 - [16] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
 - [17] F. Iandola, M. Moskewicz, S. Karayev, R. Girshick, T. Darrell, and K. Keutzer, “Densenet: Implementing efficient convnet descriptor pyramids,” *arXiv preprint arXiv:1404.1869*, 2014.
 - [18] J. Puigcerver, “Are multidimensional recurrent layers really necessary for handwritten text recognition?,” in *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, vol. 1, pp. 67–72, IEEE, 2017.
 - [19] A. F. de Sousa Neto, B. L. D. Bezerra, A. H. Toselli, and E. B. Lima, “Htr-flor: a deep learning system for offline handwritten text recognition,” in *2020 33rd SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*, pp. 54–61, IEEE, 2020.
 - [20] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the 23rd international conference on Machine learning*, pp. 369–376, 2006.
 - [21] K. Hwang and W. Sung, “Character-level incremental speech recognition with recurrent neural networks,” in *2016 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 5335–5339, IEEE, 2016.
 - [22] T. Developers, “TensorFlow,” May 2021.
 - [23] S. van der Walt, S. C. Colbert, and G. Varoquaux, “The numpy array: A structure for efficient numerical computation,” *Computing in Science Engineering*, vol. 13, no. 2, pp. 22–30, 2011.
 - [24] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, “Albumentations: fast and flexible image augmentations,” *Information*, vol. 11, no. 2, p. 125, 2020.
 - [25] T. Carneiro, R. V. Medeiros Da Nóbrega, T. Nepomuceno, G.-B. Bian, V. H. C. De Albuquerque, and P. P. R. Filho, “Performance analysis of google colab as a tool for accelerating deep learning applications,” *IEEE Access*, vol. 6, pp. 61677–61685, 2018.
 - [26] V. I. Levenshtein *et al.*, “Binary codes capable of correcting deletions, insertions, and reversals,” in *Soviet physics doklady*, vol. 10, pp. 707–710, Soviet Union, 1966.